

Understanding How USART Works

USART Communication System



MikroDuino

Amer Iqbal Qureshi | Mikrotronics Pakistan

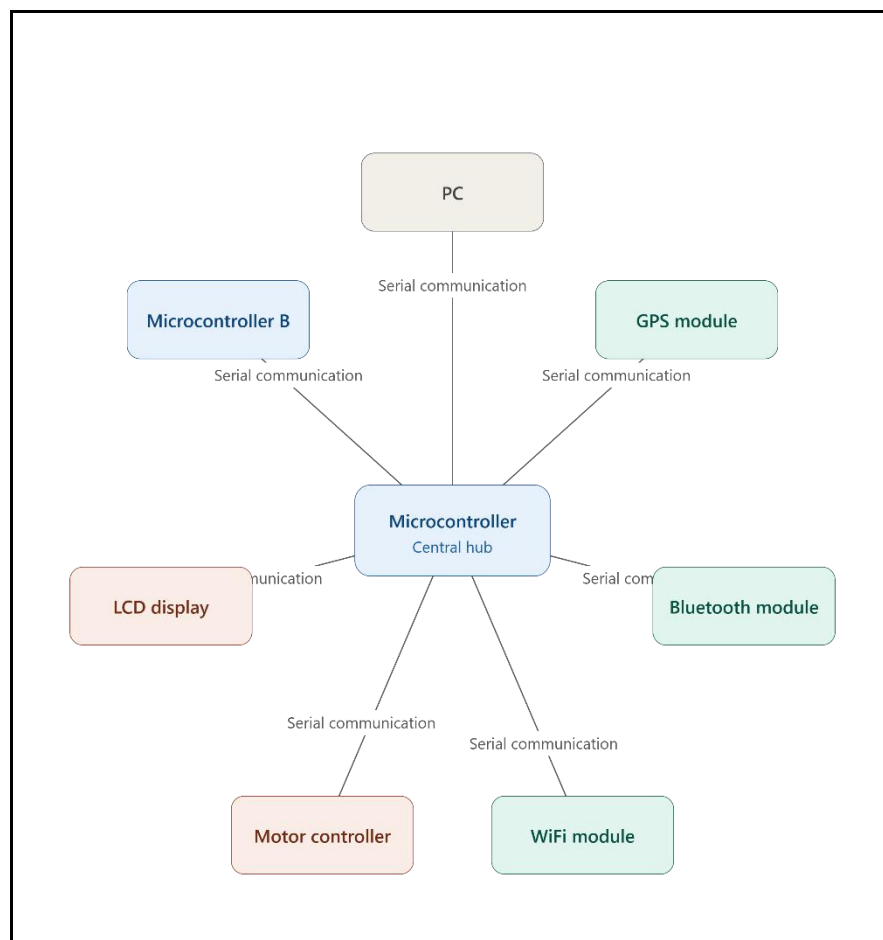
Universal Synchronous and Asynchronous Receiver/Transmitter (USART)

Speaking the Language of Machines

Imagine sitting in a room with another person. If both of you remain silent, no ideas can be exchanged. The moment one person begins to speak and the other listens, communication begins. In many ways, electronic devices behave similarly. A microcontroller, no matter how powerful, accomplishes very little if it cannot exchange information with the outside world.

Every embedded system communicates. A temperature controller reports measured temperatures to a display. A GPS receiver continuously sends location information to a navigation system. A Bluetooth module relays commands from a smartphone, while a Wi-Fi module exchanges data with servers across the Internet. Even the simple act of uploading a program from your computer to an Arduino board depends on communication between two electronic devices.

Communication, therefore, is not an optional feature of a microcontroller—it is one of its most **fundamental capabilities**. Without it, each device would exist in complete isolation, unable to share data, receive commands, or cooperate with other electronic systems.



Over the years, engineers have developed many methods of allowing digital devices to communicate. Some methods are designed for very high speeds, others for long distances, while some prioritize simplicity and reliability. Despite this diversity, one communication technique has remained remarkably popular for more than four decades: **serial communication**.

From Parallel Wires to a Single Data Line

When digital computers first appeared, it seemed natural to transmit data in parallel. Since a byte contains eight bits, engineers simply connected eight wires between two devices. Every wire carried one bit, allowing the entire byte to be transferred simultaneously.

At first glance, this appears to be the ideal solution. Eight bits travel at once, making communication seem extremely fast. However, as computers became more sophisticated and communication distances increased, several practical problems emerged.

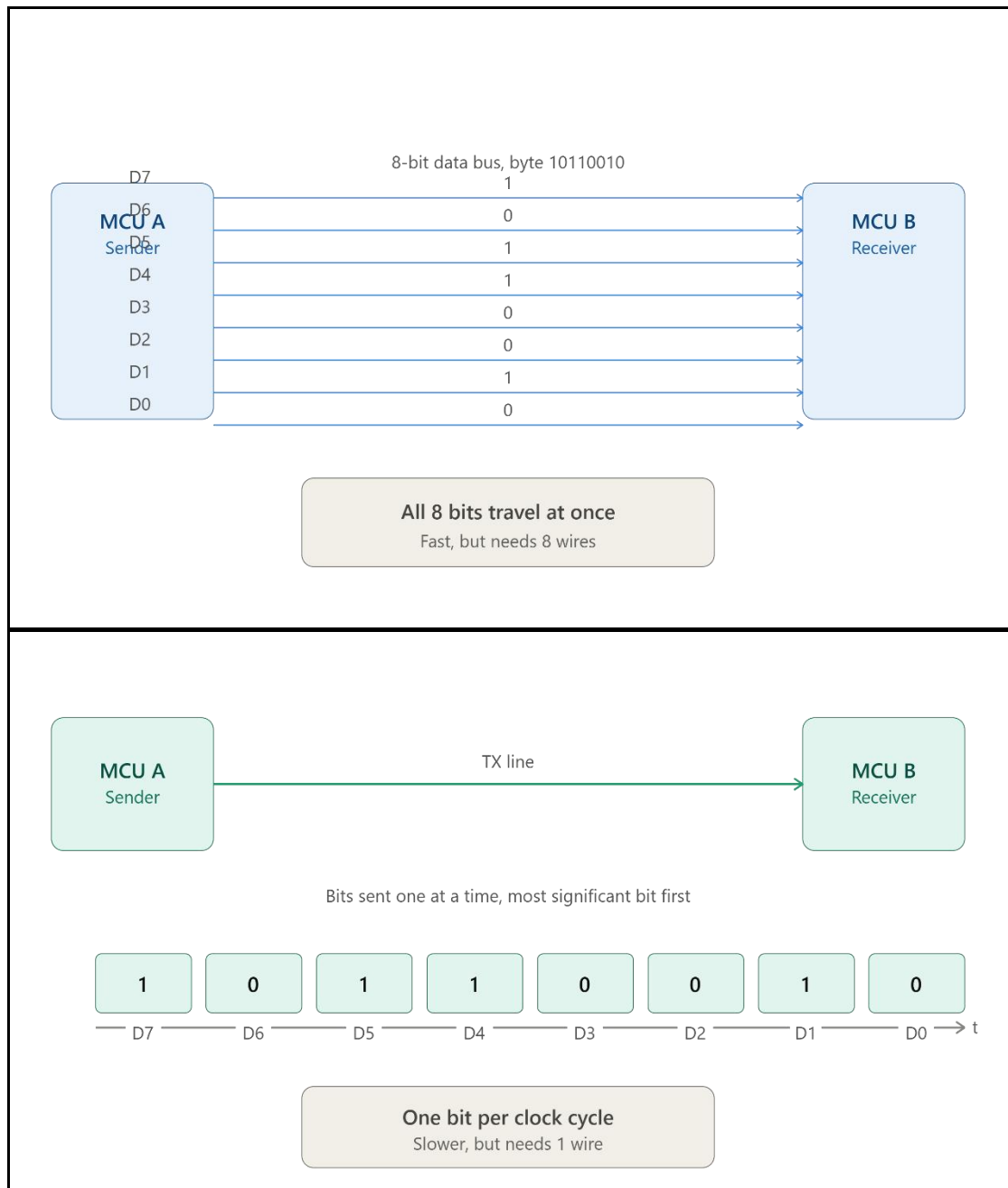
Imagine trying to keep eight people walking perfectly side by side over a long distance. Even if they all start together, some will inevitably move slightly faster or slower than the others. By the time they reach their destination, they are no longer perfectly aligned.

Electrical signals behave in a similar way. Tiny differences in wire length, electrical resistance, and capacitance cause signals to arrive at slightly different times. At low speeds these differences are insignificant, but at higher frequencies they become a serious problem. The receiving device may read some bits too early and others too late, resulting in corrupted data.

Another disadvantage is obvious the moment you examine the wiring. Eight data lines require eight PCB traces, larger connectors, thicker cables, and more pins on every integrated circuit. As systems become more complex, this quickly increases both manufacturing cost and design complexity.

Engineers eventually realized that instead of sending all eight bits simultaneously, they could send them one after another over a single wire. Although each individual byte would take slightly longer to transmit, improvements in electronic circuitry meant that bits could be sent at incredibly high speeds. The reduction in wiring more than compensated for the slight delay.

This idea gave birth to **serial communication**, a technique that today forms the foundation of almost every modern communication standard.



The Rise of Serial Communication

Today, serial communication is everywhere.

Whenever you connect an Arduino board to your computer, serial communication is taking place. When a GPS receiver reports its position, the data usually arrives over a serial interface. Bluetooth modules, GSM modems, Wi-Fi modules, industrial controllers, laboratory instruments, medical equipment, and countless sensors all rely upon some form of serial communication.

Even communication protocols that appear completely different internally are often based on the same simple principle: information travels sequentially, one bit after another.

One of the reasons serial communication became so successful is its remarkable efficiency. A single pair of communication wires can transfer thousands or even millions of bits every second. Fewer wires mean simpler circuit boards, smaller connectors, lower manufacturing costs, improved reliability, and better resistance to electrical noise.

For embedded systems, these advantages are invaluable. Every unnecessary wire occupies precious space on a printed circuit board, increases production costs, and introduces another possible point of failure. By reducing the number of required connections, serial communication allows engineers to design systems that are both simpler and more dependable.

Because of these advantages, nearly every microcontroller manufactured today includes dedicated hardware to support serial communication.

The AVR family is no exception.

The USART: A Dedicated Communication Engine

Although it is possible for a microcontroller to generate serial data entirely through software, doing so would require the processor to monitor every individual bit with extremely precise timing. During transmission, the CPU would have little opportunity to perform any other useful work.

Instead, AVR microcontrollers contain a specialized hardware peripheral known as the **Universal Synchronous and Asynchronous Receiver/Transmitter**, or **USART**.

Rather than asking the CPU to manage every electrical transition on the communication pins, the USART takes over this responsibility automatically. The software simply provides the data to be transmitted, and the USART handles the complex timing behind the scenes.

It generates the start bit, shifts each data bit onto the transmission line, optionally calculates and transmits a parity bit, appends the stop bit, and ensures that every bit is transmitted with accurate timing determined by the selected baud rate.

Reception is equally automatic. The USART continuously monitors the receive pin, detects the arrival of a start bit, samples each incoming bit at precisely calculated intervals, reconstructs the original byte, checks for communication errors, and finally informs the processor that a new character has arrived.

To the programmer, serial communication often appears as simple as reading from or writing to a register. Behind that apparent simplicity lies an impressive collection of digital logic working continuously in the background.

In many ways, the USART can be thought of as a dedicated communications assistant. While the CPU concentrates on running the main application, the USART quietly

manages the details of transmitting and receiving data, ensuring reliable communication without constant software supervision.

Understanding UART and USART

As we begin exploring the communication hardware of the AVR microcontroller, it is important to understand two terms that frequently appear in embedded systems literature: **UART** and **USART**. Although they are often used interchangeably, they are not exactly the same. Understanding the distinction between them helps in appreciating the capabilities of the communication hardware built into AVR microcontrollers.

What is a UART?

UART stands for **Universal Asynchronous Receiver/Transmitter**. It is a dedicated hardware peripheral designed to transmit and receive serial data using **asynchronous communication**.

In asynchronous communication, the transmitting and receiving devices do not share a common clock signal. Instead, both devices agree beforehand on a communication speed, known as the **baud rate**. Each device generates its own timing independently, allowing them to exchange data without requiring an additional clock wire.

A UART is responsible for all the low-level tasks involved in serial communication. When transmitting data, it converts an 8-bit (or 9-bit) parallel value from the microcontroller into a stream of serial bits. When receiving data, it performs the reverse operation by collecting the incoming serial bits and reconstructing them into a complete data byte.

Because the UART handles these operations entirely in hardware, the processor is relieved from the precise timing required to send or receive individual bits.

What is a USART?

USART stands for **Universal Synchronous and Asynchronous Receiver/Transmitter**.

As its name suggests, a USART is an enhanced version of a UART. It supports not only **asynchronous communication**, but also **synchronous communication**, making it a more flexible communication peripheral.

The AVR microcontrollers used throughout this book include a USART rather than a simple UART. This allows the same hardware module to operate in different communication modes depending on the requirements of the application.

Fortunately for beginners, most embedded applications—including those developed with Arduino—use the USART in **asynchronous mode**. Consequently, many programmers casually refer to the AVR USART as a UART because they never use its synchronous capabilities.

Asynchronous Communication

Asynchronous communication is by far the most common operating mode of the USART.

In this mode, only two signal lines are required:

- **TX (Transmit)** – carries data from the transmitting device.
- **RX (Receive)** – carries data to the receiving device.

There is **no separate clock signal** connecting the two devices. Instead, both devices are configured to operate at the same baud rate before communication begins.

To ensure that the receiver correctly identifies each transmitted byte, every data frame begins with a **start bit** and ends with one or more **stop bits**. These additional bits allow the receiver to synchronize itself with the incoming data stream.

The simplicity of asynchronous communication makes it ideal for connecting microcontrollers to computers, GPS receivers, Bluetooth modules, GSM modems, serial displays, and countless other peripherals.

Synchronous Communication

In synchronous communication, both devices share a **common clock signal**.

Along with the transmit and receive lines, an additional clock line provides precise timing information. Each transmitted bit is sampled according to this shared clock, eliminating the need for start and stop bits for every character.

Because both devices remain continuously synchronized, synchronous communication can achieve higher data rates and improved timing accuracy compared with asynchronous communication.

Although synchronous mode is supported by the AVR USART, it is used far less frequently than asynchronous mode in typical embedded applications.

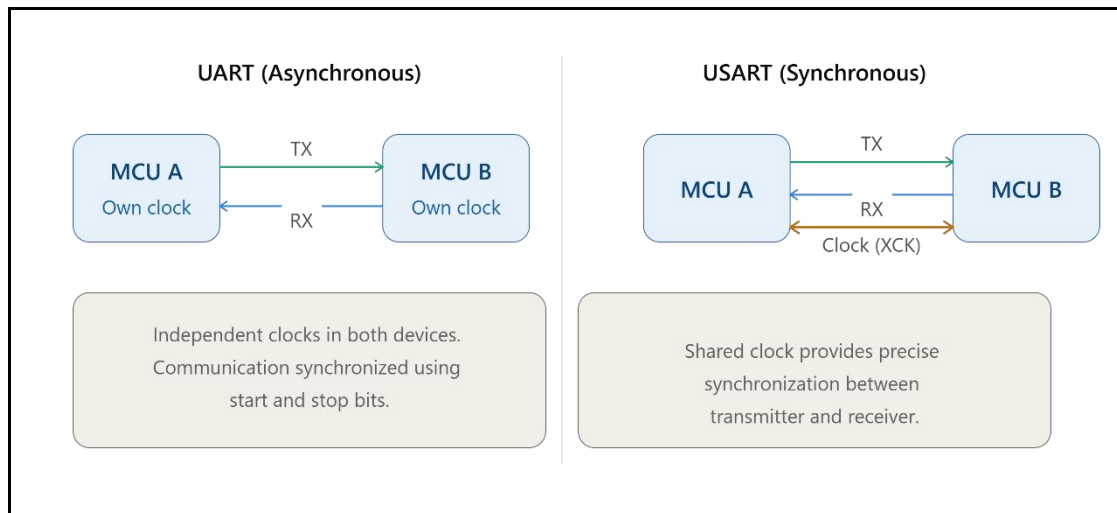
UART vs. USART

The relationship between UART and USART can be summarized simply:

- A **UART** supports **only asynchronous communication**.
- A **USART** supports **both asynchronous and synchronous communication**.

In other words, every USART can function as a UART, but a UART cannot operate as a USART because it lacks the hardware required for synchronous communication.

This distinction explains why datasheets for AVR microcontrollers consistently refer to the peripheral as a **USART**, while many software libraries and online tutorials simply call it a **UART**.



Hardware Overview of the AVR USART

At first glance, serial communication appears surprisingly simple. A programmer writes a character into a register, and a few moments later that character appears on another device. Likewise, when another device sends data, a character magically appears in another register, ready to be read by the program.

This apparent simplicity hides a remarkable amount of hardware working quietly behind the scenes.

Imagine being asked to send a handwritten letter. Before the letter reaches its destination, several steps must occur. You must write the message, place it in an envelope, write the recipient's address, seal it, hand it to the postal service, and finally wait for it to be delivered. At the receiving end, someone opens the envelope, reads the letter, and passes the message to its intended recipient.

The AVR USART performs a remarkably similar sequence of operations every time a byte is transmitted or received. It acts as an intelligent communications assistant, handling all the precise timing and formatting required for reliable serial communication, allowing the CPU to concentrate on running the application.

Throughout this section, we shall follow the journey of a single character as it travels through the USART hardware. By understanding the role of each internal component, the operation of the USART becomes much easier to understand and program.

The USART at a Glance

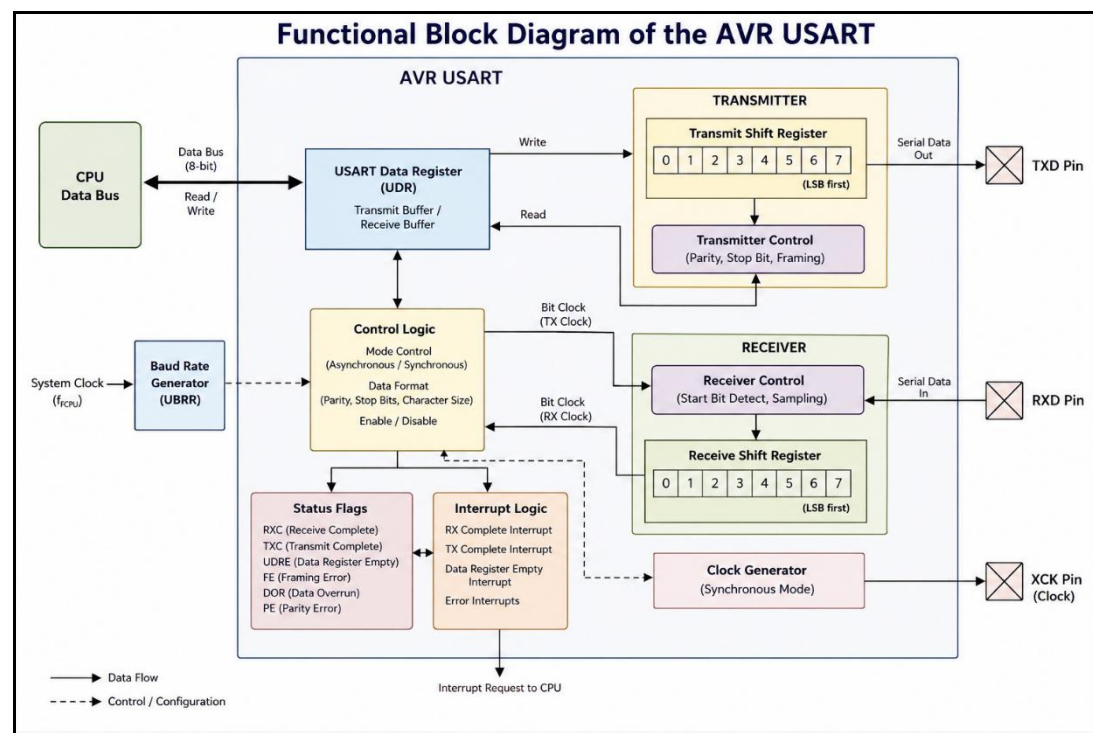
Although the USART appears to the programmer as only a handful of registers, internally it consists of several specialized hardware blocks working together.

Some blocks are responsible for generating accurate timing. Others convert parallel data into serial form. Some detect communication errors, while others notify the processor when attention is required.

The major functional blocks include:

- Baud Rate Generator
- Transmitter
- Receiver
- Shift Registers
- Data Buffer Registers
- Control Logic
- Status Logic
- Interrupt Controller
- Clock Generator (Synchronous Mode)

Each performs a specific task, yet all operate simultaneously to provide smooth and reliable communication.



The Journey of a Character

Suppose our program wishes to transmit the letter 'A'. In ASCII, the character 'A' is represented by the binary value:

01000001

When the program writes this value into the USART Data Register (UDR), the work of the CPU is essentially complete.

From this point onward, the USART hardware takes control.

It prepares the data for transmission, converts the eight bits into a serial stream, precisely controls the timing of every transmitted bit, and finally signals the processor that it is ready for another byte.

Receiving data follows the opposite path.

As electrical signals arrive on the RX pin, the USART reconstructs the original byte, checks it for errors, stores it safely in the receive buffer, and informs the processor that a character has arrived.

This automatic handling is what makes hardware serial communication both reliable and efficient.

The Baud Rate Generator

Every form of digital communication depends upon accurate timing. If one device transmits bits faster than the receiving device expects, the received data will become corrupted.

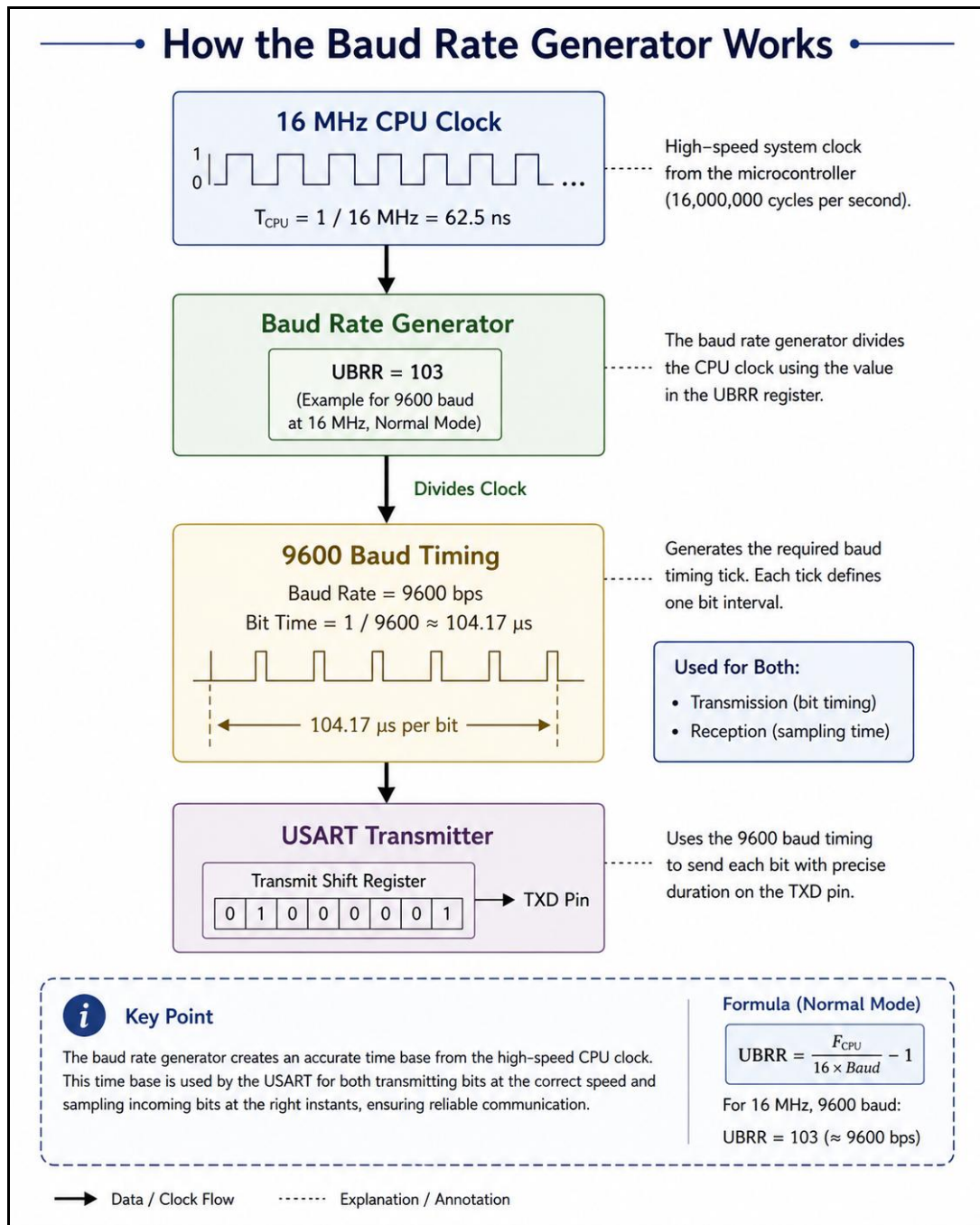
To prevent this, the USART includes a dedicated **Baud Rate Generator**. Its purpose is to derive a precise communication clock from the much faster system clock supplied to the microcontroller.

Consider an ATmega328P running at 16 MHz. The processor executes sixteen million clock cycles every second. However, if the USART is configured for a baud rate of 9600 bits per second, each transmitted bit must occupy approximately:

```
1 / 9600
≈104 microseconds
```

Clearly, the USART cannot simply use the processor clock directly. Instead, the Baud Rate Generator divides the high-speed clock into precisely timed intervals suitable for serial communication.

The division ratio is determined by the **UBRR (USART Baud Rate Register)**, which allows the programmer to select almost any standard communication speed. Because this timing is produced entirely by dedicated hardware, the processor does not need to count clock cycles or generate delays in software.



The Transmitter

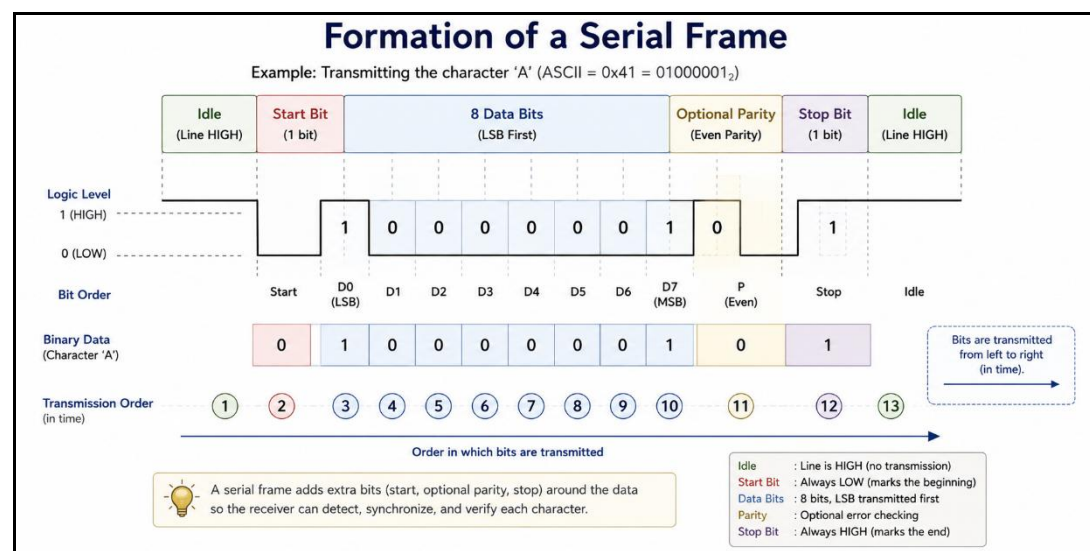
Once the baud rate has been established, the USART is ready to send data. The programmer simply writes a byte into the USART Data Register. The transmitter then performs several operations automatically.

It waits until the communication line is idle. It generates a **start bit**, informing the receiving device that a new character is about to begin.

Next, it sends each data bit individually, **beginning with the least significant bit**. Depending upon the configuration, it may also transmit a parity bit.

Finally, it appends one or two stop bits before returning the communication line to its idle state. All of these operations occur entirely in hardware.

No software intervention is required after the byte has been written into the data register.

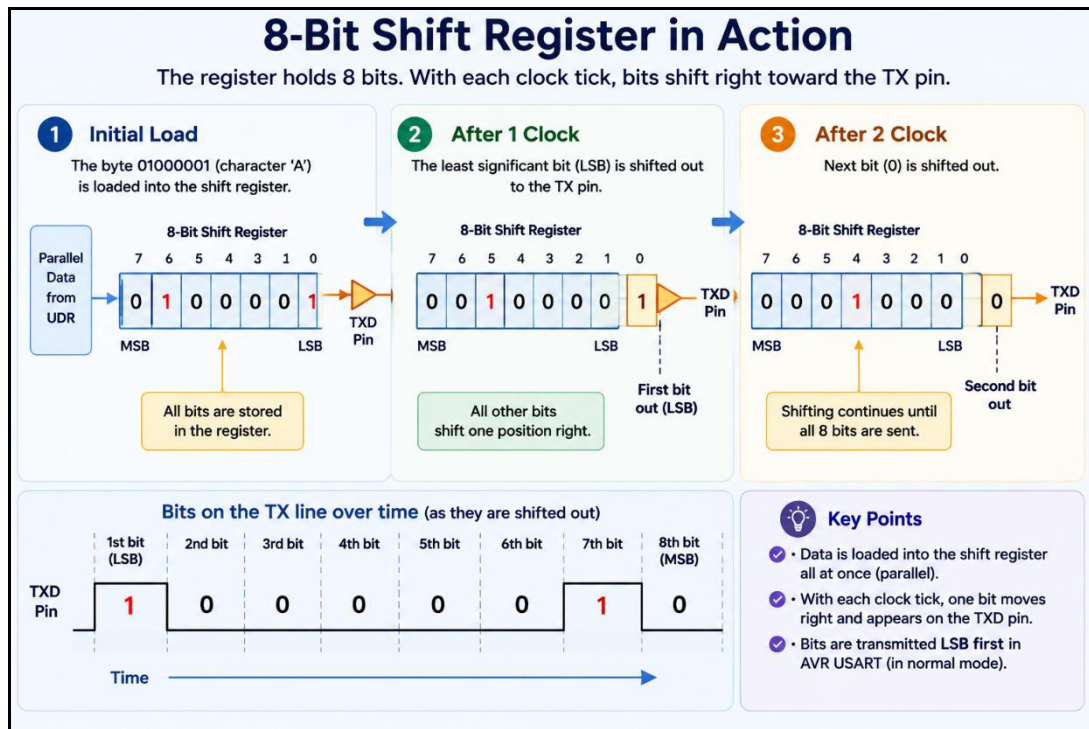


The Transmit Shift Register

The processor stores information eight bits at a time. Communication lines, however, can carry only one bit at a time. The Transmit Shift Register serves as the bridge between these two worlds.

Once loaded with a complete byte, the shift register gradually shifts one bit onto the TX pin during each baud clock interval. After one bit has been transmitted, the remaining bits move one position closer to the output.

This process continues until every bit has been sent. While this is happening, the processor is free to perform entirely unrelated tasks. This ability to operate independently is one of the greatest advantages of hardware peripherals.



The Receiver

Receiving data is considerably more challenging than transmitting it. Unlike the transmitter, which controls the timing of every transmitted bit, the receiver has no advance knowledge of when another device will begin sending data. Consequently, the receiver continuously monitors the RX pin.

When it detects a transition from logic HIGH to logic LOW, it recognizes the arrival of a **start bit**. This single transition informs the USART that an incoming character has begun.

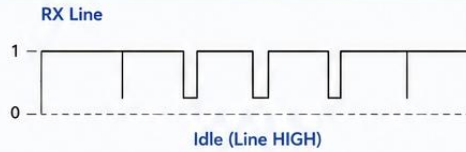
The receiver immediately starts its internal timing circuitry. Using the baud rate generator, it samples the communication line at precisely calculated intervals. Each sampled bit is stored inside the Receive Shift Register.

After all data bits have been collected, the receiver checks the stop bit. If everything appears correct, the completed byte is transferred into the receive buffer. Only then is the processor informed that new data has arrived.

UART Receiver Operation – From RX Line to CPU

1 IDLE LINE

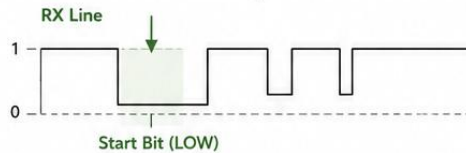
RX line is normally HIGH (logic '1'). The receiver continuously monitors the line.



Receiver is waiting for activity on RX line.

2 START BIT DETECTED

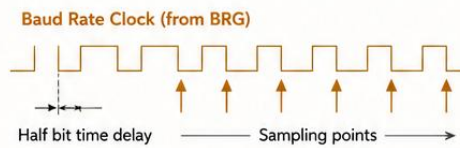
RX line goes LOW. This HIGH-to-LOW transition indicates the start of a new frame.



Start bit detected. Receiver recognizes the beginning of a new character.

3 INTERNAL TIMER STARTS

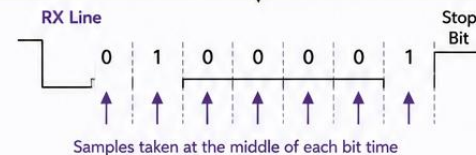
The receiver starts its internal timer using the baud rate clock. It waits half a bit time to sample in the middle of the start bit.



Timer synchronizes the receiver to the incoming data.

4 SAMPLE DATA BITS

The receiver samples the RX line at the center of each bit time and stores the bits (LSB first).



Each bit is sampled at the optimal time for maximum reliability.

5 RECEIVE SHIFT REGISTER

Each sampled bit is shifted into the Receive Shift Register (LSB first).



Bits are assembled serially into a full 8-bit character.

6 RECEIVE BUFFER

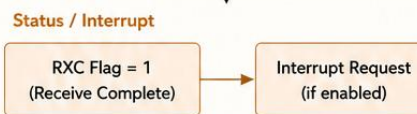
After the stop bit is verified, the complete byte is transferred to the Receive Buffer (UDR).



The complete byte is stored safely in the buffer.

7 CPU NOTIFICATION

The receiver sets the RXC (Receive Complete) flag and/or generates an interrupt (if enabled).



The CPU is informed that a new character has arrived.

8 CPU READS DATA

The CPU reads the byte from the Receive Buffer (UDR) and clears the RXC flag.



The character is now available to the application.

Example: The received character is 'A' (ASCII = 0x41 = 01000001₂)

↑ Sampling point ↓ Event / Transition

The Receive Shift Register

The Receive Shift Register performs the opposite function of the Transmit Shift Register. Instead of shifting bits outward, it gradually assembles incoming bits into a complete byte.

Each correctly timed sample occupies one position within the register. After the final data bit has been received, the shift register transfers the completed byte into the USART Data Register.

Only after this transfer can the processor safely read the received character.

Double Buffering

One particularly elegant feature of the AVR USART is its use of **double buffering**.

Suppose the transmitter is still busy sending one character. Must the processor wait until transmission finishes before supplying another character?

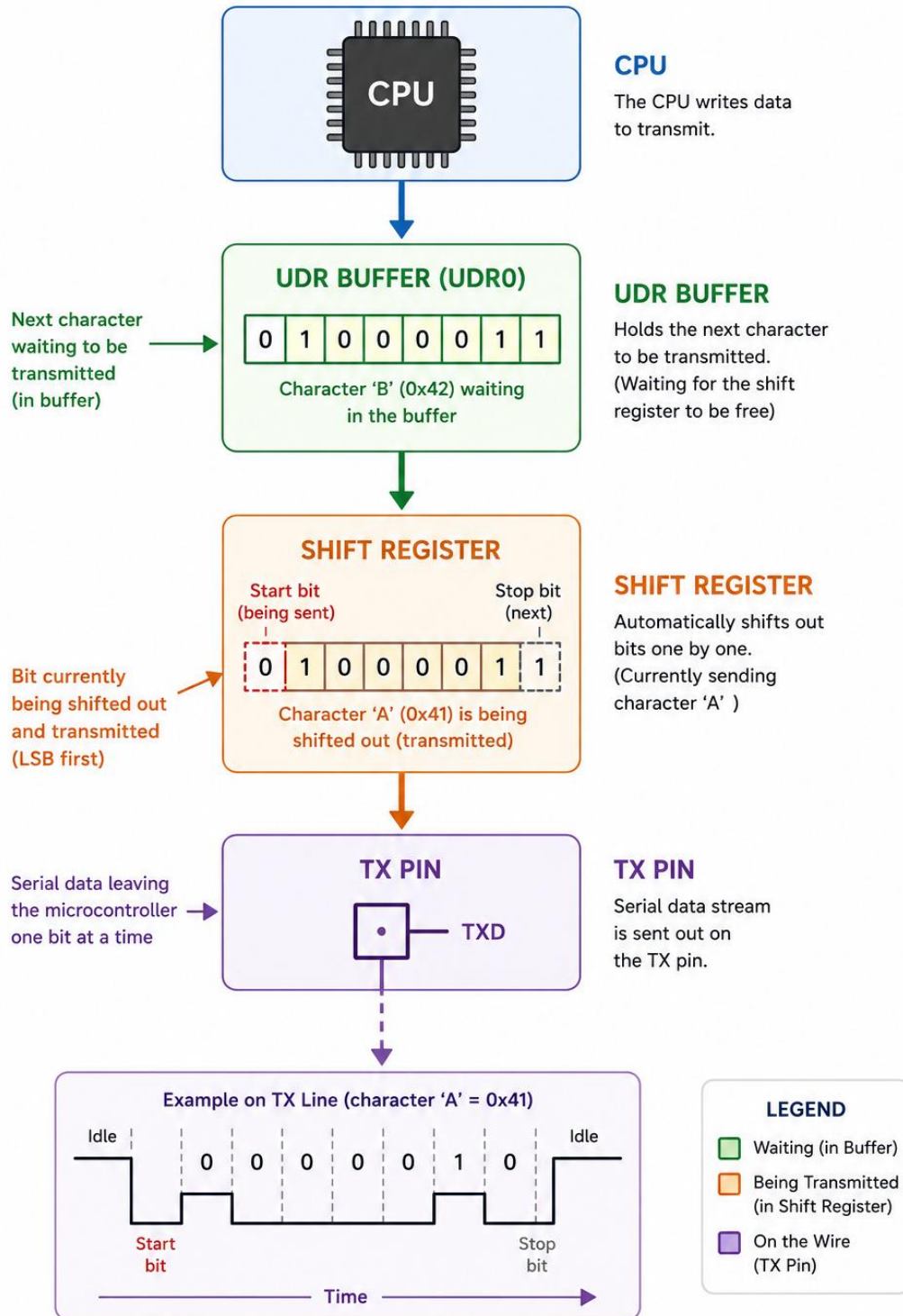
Fortunately, the answer is no.

The USART includes an intermediate buffer. While one character is actively being shifted onto the TX line, another character can already be waiting inside the Data Register. As soon as the first transmission completes, the second character is transferred automatically into the shift register.

This overlapping operation allows characters to be transmitted continuously without unnecessary gaps.

Double buffering significantly increases communication efficiency.

UART TRANSMIT PATH



Error Detection Logic

Reliable communication requires more than simply sending and receiving bits. The USART continuously checks whether communication is occurring correctly. Among the errors it can detect are:

Framing Error

Occurs when the expected stop bit is not detected. This often indicates incorrect baud rate settings or excessive electrical noise.

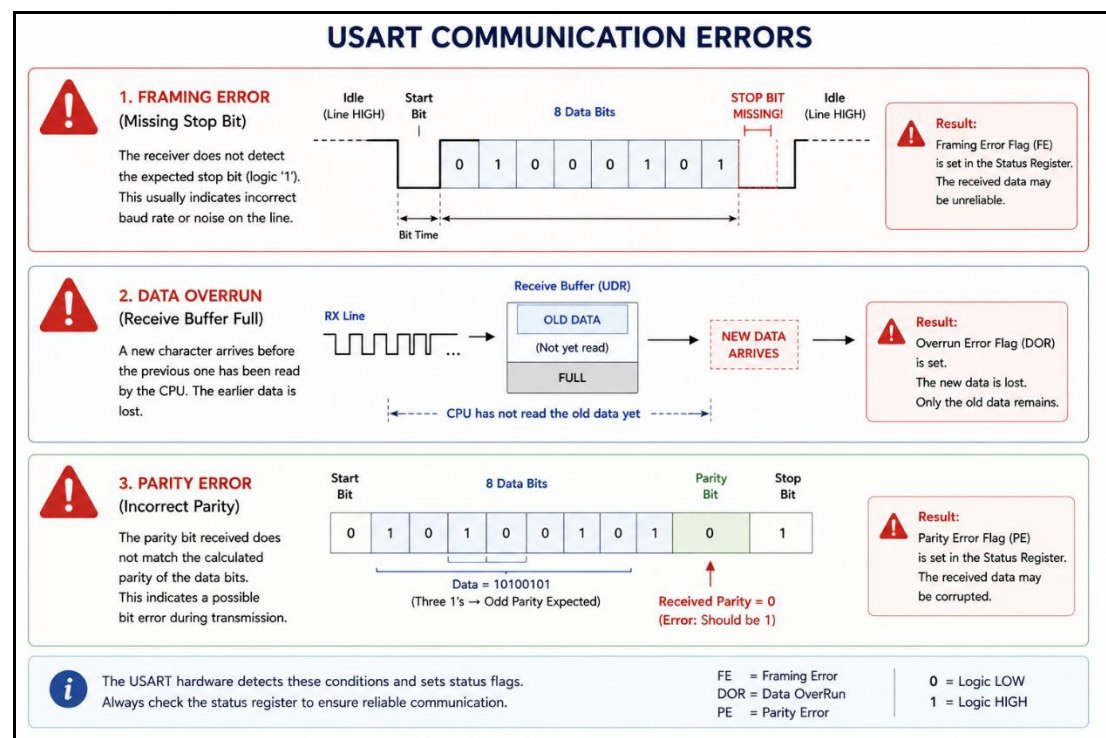
Data Overrun

Occurs when a new character arrives before the previous character has been read by the processor. The earlier character is lost.

Parity Error

Occurs when the received parity bit does not agree with the calculated parity. Although parity is not always used, it provides a simple method of detecting single-bit transmission errors.

These error conditions are recorded in status flags that software can examine.



Status Logic

The USART continuously informs the processor about its internal condition. Rather than constantly interrupting the CPU, it maintains a collection of status flags.

These indicate events such as:

- Transmit buffer empty
- Transmission complete
- Character received
- Communication errors

The processor can simply inspect these flags whenever necessary. This approach, known as **polling**, is often sufficient for simple embedded applications.

Later in this chapter, we shall see how these same events can instead generate interrupts, allowing the CPU to perform other tasks until communication requires attention.

Interrupt Logic

In many applications, constantly checking status flags wastes valuable processor time. Instead, the USART can generate hardware interrupts whenever important communication events occur.

For example:

- A new character has been received.
- The transmit buffer has become empty.
- Transmission of the final stop bit has completed.

When one of these events occurs, the USART temporarily interrupts normal program execution and transfers control to an Interrupt Service Routine (ISR), where the event can be handled immediately.

Interrupt-driven communication enables highly efficient applications, particularly when communication occurs frequently or at high data rates.

Clock Generator (Synchronous Mode)

Although asynchronous communication is used in the vast majority of embedded applications, the AVR USART also supports synchronous communication.

In this mode, an additional signal called **XCK (External Clock)** provides timing information for every transmitted bit.

When configured as a master, the AVR generates this clock and supplies it to the receiving device. When operating as a slave, it accepts the clock from an external master.

Because both devices share exactly the same timing reference, synchronous communication can achieve higher throughput and more predictable timing than asynchronous communication.

Bringing Everything Together

Although we have examined each hardware block individually, they never operate in isolation. Every time a single character is transmitted or received, all of these components work together as a coordinated system.

A byte written by the CPU enters the data buffer, is transferred to the transmit shift register, timed by the baud rate generator, and sent bit by bit through the TX pin. At the receiving end, the reverse process takes place: the RX pin detects the incoming signal, the baud rate generator determines when to sample each bit, the receive shift register reconstructs the original byte, error detection logic verifies its integrity, and the completed character is placed in the receive buffer before notifying the processor.

Understanding this journey is the key to mastering the AVR USART. In the following sections, we will move from the internal hardware to the programmer's perspective, learning how to configure the USART registers, calculate baud rates, and write efficient software that makes full use of this powerful communication peripheral.